

Trabalho Prático 01

Calculadora de Conjuntos em GNU Octave

Disciplina: Matemática Discreta
Curso: Bacharelado em Sistemas de Informação
Professor: Eduardo Max Amaro Amara
Instituição: IFES - Campus Colatina
Data Limite de Entrega: 24 de agosto

1. Objetivos do Trabalho

O objetivo deste trabalho prático é aplicar os fundamentos da **Teoria e Álgebra de Conjuntos** na construção de uma ferramenta computacional estruturada e modular em **GNU Octave**. Os principais objetivos consistem em:

- Representar computacionalmente o conceito formal de conjunto e suas propriedades fundamentais.
- Implementar um algoritmo modularizado capaz de manipular conjuntos finitos de números inteiros (Calculadora de Conjuntos).
- Consolidar a criação de rotinas modulares através de arquivos de função (.m), respeitando o escopo local de variáveis.
- Praticar o tratamento de dados (como eliminação de elementos duplicados) e operações de pertinência.

2. Especificação de Entrada

ENTRADA Leitura e Formato dos Dados

O programa deverá receber como entrada dois conjuntos: A e B , de tamanhos n e m respectivamente, onde $A, B \subseteq \mathbb{Z}$ (conjuntos formados por números inteiros informados pelo usuário).

- O programa deve ser capaz de receber essas entradas de forma interativa e tratar eventuais repetições inseridas, garantindo a unicidade dos elementos de cada conjunto antes de processar as operações.

3. Funcionamento do Programa e Operações Requeridas

O sistema deverá disponibilizar um menu interativo ou fluxo principal permitindo a execução das seguintes operações fundamentais da álgebra de conjuntos:

OPERAÇÕES Lista de Funcionalidades Obrigatórias

1.

Inserir/Alterar os conjuntos A e B : Leitura dos vetores de números inteiros fornecidos pelo usuário.

2. **União ($A \cup B$):** Gera o conjunto formado por todos os elementos que pertencem a A , a B ou a ambos, sem repetições.

3. **Intersecção ($A \cap B$):** Gera o conjunto formado apenas pelos elementos comuns a ambos

os conjuntos.

4. **Diferença de A por B ($A - B$):** Gera o conjunto dos elementos que pertencem exclusivamente a A e não pertencem a B .
5. **Diferença de B por A ($B - A$):** Gera o conjunto dos elementos que pertencem exclusivamente a B e não pertencem a A .
6. **Diferença Simétrica ($A \Delta B$):** Retorna todos os elementos que pertencem exclusivamente a um dos dois conjuntos ($A \Delta B = (A - B) \cup (B - A)$).
7. **Produto Cartesiano ($A \times B$):** Gera e exibe o conjunto de todos os pares ordenados (a, b) tais que $a \in A$ e $b \in B$.

4. Padrão de Modularização e Exemplo de Código

Para garantir boas práticas de desenvolvimento de software e clareza de escopo, cada operação e rotina de suporte deve ser implementada em arquivos .m individuais de função. Abaixo apresenta-se o padrão esperado utilizando a operação de **União** ($A \cup B$) como exemplo:

1. Função Auxiliar: `pertence.m`

Esta subfunção verifica se um determinado elemento já existe dentro de um vetor/conjunto, servindo de suporte para evitar duplicatas e realizar buscas:

```
function resultado = pertence(elemento, conjunto)
    % Retorna true (1) se o elemento estiver no conjunto, ou false
    (0) caso contrario
    resultado = false;

    for i = 1:length(conjunto)
        if conjunto(i) == elemento
            resultado = true;
            return; % Encerra a busca assim que encontrar
        endif
    endfor
endfunction
```

2. Função Principal: `uniao_conjuntos.m`

A função de união instancia o conjunto resultante com os elementos de A e concatena apenas os elementos de B que não estejam presentes no resultado:

```
function C = uniao_conjuntos(A, B)
    % Inicializa o conjunto resultante com os elementos do conjunto A
    C = A;

    % Percorre cada elemento do conjunto B
    for i = 1:length(B)
```

```

        % Se o elemento de B nao estiver presente no conjunto C,
adiciona-o
        if ~pertence(B(i), C)
            C = [C, B(i)]; % Concatena o novo elemento ao vetor
        endif
    endfor
endfunction

```

Diretrizes de Avaliação de Código:

- **Modularização Estrita:** Implementação de funções auxiliares e principais em arquivos .m individuais.
- **Escopo Local:** Nenhum cálculo interno deve depender de variáveis globais do *Workspace*.
- **Validação Formal:** As saídas devem respeitar rigorosamente a definição matemática de conjunto (sem elementos repetidos).

5. Diretrizes do Relatório e Entrega

ENTREGA Estrutura do Documento e Normas

Deverá ser entregue um **relatório técnico em formato PDF** contendo obrigatoriamente:

1. **Estratégia Utilizada:** Descrição formal da lógica e estratégias adotadas para cada uma das operações de conjuntos solicitadas.
2. **Código-fonte e Estruturas:** Apresentação comentada dos algoritmos implementados, funções desenvolvidas e estruturas de dados utilizadas.
3. **Demonstração de Testes:** Exemplos de execução das operações com casos de teste representativos (incluindo casos com conjuntos disjuntos, conjuntos com elementos em comum e conjunto vazio).

Modalidade e Ferramenta:

- O trabalho pode ser realizado de forma **individual ou em dupla**.
- A implementação deve ser desenvolvida integralmente em **GNU Octave**.
- **Prazo Final de Envio:** até **24 de agosto**.